AetherGovern: A Falsification-First Investigation of Governance in the Aether Family

A Falsification-First Investigation of Authority, Policy, and Accountability in Multi-Agent Software Ecosystems

Samir Baladi

Interdisciplinary AI Research — Ronin Institute / Rite of Renaissance

Research Article — Distributed Systems / Software Engineering / Institutional Analysis

DOI: https://doi.org/10.5281/zenodo.24054524

ORCID: 0009-0003-8903-0029 · gitdeeper@gmail.com

Version 0.1.0 — November 2026

Status: *Architectural Hypothesis Under Investigation (v0.1.0). This is not a committed project. It is a structured investigation of whether a governance gap genuinely exists in the Aether family. No implementation is claimed. No real-world deployment has been evaluated. The final project structure (single, dual, or triple project) will be determined after comparison.*

Abstract

The Aether family comprises four preprints that address, respectively, how agent-generated changes are stored (AetherGit), integrated and verified operationally (AetherFlow), specified and formally evidenced (AetherProof), and synchronized without a central coordinator (AetherMesh). None of them is primarily about *authority*: who may act, under which policy, with what accountability, and how authority is withdrawn. This paper investigates whether that omission is a genuine gap that warrants a fifth functional dimension, tentatively named AetherGovern, or merely an artifact of how the existing dimensions were scoped. It does so with a falsification-first method: before describing the candidate, it states five failure conditions (F1–F5) whose satisfaction would show that the candidate does not deserve to exist as a separate dimension, together with a decision rule. It then compares three candidate capabilities (orchestration, governance, and runtime assurance), specifies a five-layer governance architecture (authorization, policy, accountability, revocation, dispute resolution), and states two propositions with proof sketches: that authority checks which cite their own revocation frontier preserve the determinism that AetherMesh's convergence argument requires, and that revocation in an eventually consistent setting leaves an exposure window that no decentralized design can remove. On the basis of argument alone, conditions F1–F4 are provisionally not falsified, with stated residual weaknesses, and F5 (observable benefit) is untested. The paper therefore concludes that AetherGovern is a candidate, not a confirmed dimension. No implementation, experiment, or measurement is reported.

Keywords: *Governance; Authorization; Delegation; Revocation; Accountability; Multi-Agent Systems; Falsification; Decentralized Systems; Agentic Software Engineering; AetherGovern.*

Evidentiary status: *hypothesis under investigation. Every statement in this paper is labeled as a definition, a hypothesis, or an observation. Propositions are stated with proof sketches and are not machine-checked. No empirical result is reported. Descriptions of third-party work follow the cited sources and were not independently verified.*

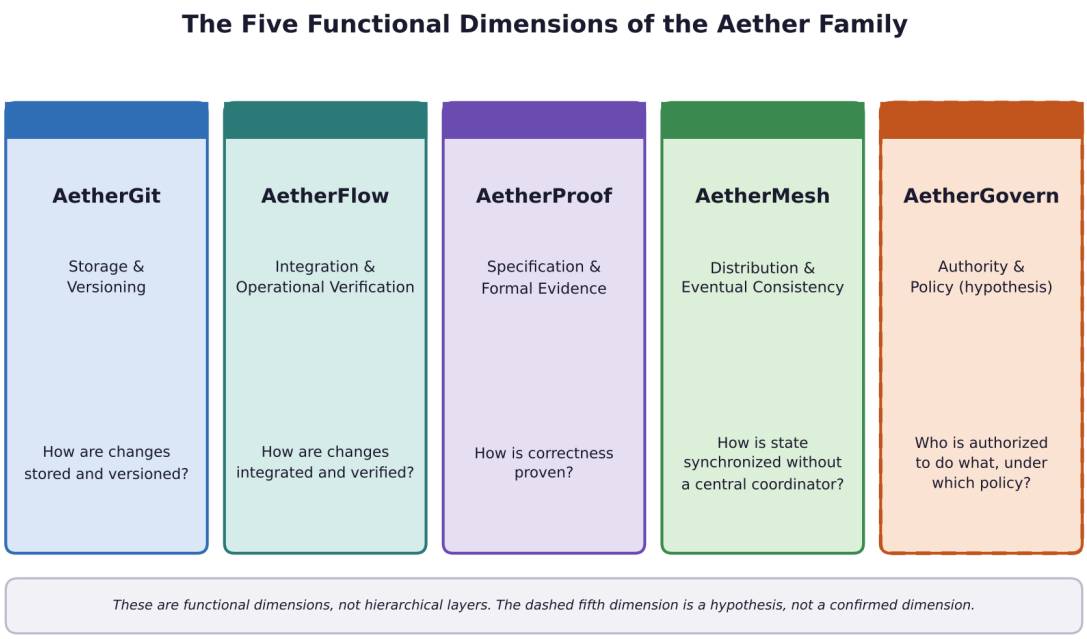


Figure 1 (Fig1_Five_Dimensions.png). The five functional dimensions of the Aether family. The fifth dimension (AetherGovern) is a hypothesis under investigation, not a confirmed dimension. The dimensions are functional, not hierarchical; the figure was redrawn for this record.

Table of Contents

1. Introduction	3
2. Background and Related Work	5
3. The Falsification-First Methodology	7
4. The Three Candidate Capabilities	10
5. AetherGovern: Architecture	11
6. AetherGovern: Interfaces	19
7. Evaluation	21
8. Conclusion and Future Work	25
Appendix A — Notation	27
Appendix B — The Five Failure Conditions in Detail	27
Appendix C — Open Questions Q1–Q20	29
Appendix D — Glossary	31
References	31
Research Status Statement	33

1. Introduction

1.1 The Coordination Bottleneck in Multi-Agent Ecosystems

Software is increasingly modified by autonomous agents that read a repository, form an intent, and submit changes with little per-change human supervision. When a handful of agents work in one repository, conventional version control and human review provide adequate coordination. As the number of agents grows, coordination itself becomes the limiting resource: every change must be stored, integrated, checked, and reconciled with concurrent changes, and the mechanisms that do this must themselves scale. The Aether preprints (Baladi, 2026a–d) treat four aspects of this problem. This paper asks whether a fifth aspect, *governance*, has been left unaddressed, and if so whether it deserves a dimension of its own.

The question is not whether governance matters in some abstract sense. It is a narrower, falsifiable question about the structure of a particular family of architecture proposals. Every multi-agent ecosystem embeds answers to questions such as: which agent may merge into which branch; who may change the rules that decide this; how an agent's authority is withdrawn when its key is stolen; who is answerable when an admitted change causes harm; and what happens when two agents hold conflicting claims to authority. In a centrally orchestrated system these answers are implicit in the orchestrator's configuration. In a decentralized system they cannot be implicit, because no node is distinguished. Whether they are already adequately specified by the existing dimensions, or whether they form a separable concern, is the object of investigation.

1.2 The Aether Family: Four Foundational Dimensions

Each of the four existing works is a preprint published on Zenodo. None has been peer reviewed, and, as far as the author's records state, none has an evaluated implementation. We summarize each in one sentence and give the question it addresses.

Dimension	Focus	Question	Record
AetherGit	Storage and versioning	How are changes stored and versioned?	Baladi (2026a), 10.5281/zenodo.23966896
AetherFlow	Integration and operational verification	How are changes integrated and verified?	Baladi (2026b), 10.5281/zenodo.23979129
AetherProof	Specification and formal evidence	How is correctness proven?	Baladi (2026c), 10.5281/zenodo.24013124
AetherMesh	Distribution and eventual consistency	How is state synchronized without a central coordinator?	Baladi (2026d), 10.5281/zenodo.24038038
AetherGovern (hypothesis)	Authority and policy	Who is authorized to do what, under which policy?	This paper, 10.5281/zenodo.24054524

The word *dimension* is chosen deliberately. Earlier descriptions of the family used the word *layer*, which suggests a vertical stack in which each layer depends on those below it. The relationships among the four works are not strictly hierarchical: AetherProof does not sit above AetherFlow in any enforcement sense, and AetherMesh can be adopted without AetherProof. We therefore describe them as functional dimensions, as in Figure 1. This is a terminological choice made for accuracy, and it has a consequence for this investigation: a candidate fifth dimension must be shown to be *functionally distinct*, not merely to occupy a further position in a stack.

1.3 The Central Question: Is Governance a Genuine Gap?

Central question *(definition).* Is there a body of authorization, policy, accountability, revocation, and dispute-resolution requirements for multi-agent software ecosystems that (i) is not already specified by the four existing dimensions, (ii) is not reducible to verification or to orchestration, (iii) can be specified without reintroducing a central authority, and (iv) yields an observable benefit over existing mechanisms?

The four clauses correspond to failure conditions F1, F2 and F3 (clause ii), F4, and F5 introduced in Section 3. A negative answer to any clause would show that governance, as a separate dimension, is not warranted. A positive answer to all four, supported by evidence, would show that it is a candidate. The paper is organized to attempt to *refute* the candidate before describing it in detail.

1.4 Contributions to the Investigation

The following are contributions to an *investigation*. They are not to be read as committed architectural contributions, and none is validated by implementation.

1. **A falsification-first procedure** with five failure conditions and a decision rule, stated before the candidate is described (Section 3 and Appendix B).
2. **A comparison of three candidate capabilities** that could fill a gap beside the four dimensions: orchestration, governance, and runtime assurance (Section 4).
3. **A hypothesized five-layer governance architecture**, including a policy lifecycle, a scoped delegation model with revocation, and a statement of how governance differs from verification (Section 5).
4. **Two propositions with proof sketches** that bear directly on F4: causal-authority admission preserves the determinism requirement of AetherMesh, and revocation leaves an irreducible exposure window (Section 5.3).
5. **A provisional assessment** of F1–F5 based on argument alone, with residual weaknesses stated, and an evaluation design that specifies in advance what outcomes would count as failure (Sections 3 and 7).
6. **Twenty open questions** (Section 7.5 and Appendix C).

1.5 Evidentiary Status

Every statement in this paper belongs to one of three kinds, and we label it accordingly where the distinction matters. A *definition* fixes a term and carries no truth claim. A *hypothesis* is a claim that we think plausible and have not established. An *observation* is a claim about the existing literature or the existing Aether records that a reader can check against the cited source. We do not use a fourth category of *result*, because no result is reported.

Item	Status
Existence of a governance gap	Hypothesis under investigation
Five failure conditions F1–F5	Definitions (proposed test criteria)
Five-layer architecture	Hypothesis; specification only
Propositions 1–2	Stated with proof sketches; not machine-checked
Assessment of F1–F4	Argument only; provisional
Assessment of F5	Untested
Implementation, deployment, measurement	None

DOI: https://doi.org/10.5281/zenodo.24054524

Item	Status
Project structure (single, dual, or triple)	Undetermined

Limitations of this section. This section defines scope and terminology. The claim that the four existing works are 'dimensions' rather than 'layers' is a terminological revision, not a verified architectural fact.

2. Background and Related Work

This section reviews the Aether family, the governance literature on which the investigation draws, and the methodological background. Descriptions of third-party work follow the cited sources and were not independently verified.

2.1 The Aether Family (Four Projects)

AetherGit (Baladi, 2026a) proposes a storage and versioning model for agent-generated changes, built around an append-only changeset log in which each changeset carries pointers to its parents. **AetherFlow** (Baladi, 2026b) proposes post-merge verification and dynamic conflict resolution, with a message-based protocol through which changes are admitted, checked, and remediated. **AetherProof** (Baladi, 2026c) proposes specification-driven proof construction: a canonical representation of specifications, a dependency graph of proof obligations and evidence, counterexample-guided remediation, and a human review protocol. **AetherMesh** (Baladi, 2026d) proposes decentralized synchronization by combining CRDTs (Shapiro et al., 2011; Almeida et al., 2018), epidemic gossip (Demers et al., 1987; Karp et al., 2000; Jelasity et al., 2005), and a lightweight Byzantine admission layer.

Four observations about these records matter here. *(Observation 1.)* AetherMesh's admission layer relies on a local key registry and states that the registry's bootstrap, rotation, and revocation are not specified (its threat T6 and open question Q31). *(Observation 2.)* AetherMesh requires that admission predicates be deterministic functions of a delta and its causal closure, because otherwise honest nodes can admit different sets and diverge (its assumption A6). *(Observation 3.)* AetherProof's human review protocol assigns roles and separation of duties for specification and proof decisions, which is a governance-like fragment confined to the verification domain. *(Observation 4.)* All four records are architecture proposals whose evidence is argument or simulation; none reports a deployed system. Together, these observations suggest that fragments of governance exist inside the existing dimensions, which is precisely why the question of redundancy (F1) must be taken seriously.

2.2 Governance in Multi-Agent Systems

Governance is a subject of long-standing study outside computer science, and we draw on that literature for framing, not for proof. In each case we state what the source argues and mark the mapping to the Aether setting as a hypothesis.

Hardin (1968) argues that individuals acting in their own interest can ruin a shared resource when access is unrestricted. The analogy to a shared codebase modified by many autonomous agents is suggestive: each agent's local incentive may be to merge quickly, while the cost of a degraded shared state is borne by all. We treat the analogy as a hypothesis motivating attention to access rules, not as evidence that the problem is severe in practice.

Ostrom (1990) argues, on the basis of empirical studies of long-enduring common-pool resource institutions, that communities can govern shared resources without either central control or privatization, and identifies recurring design principles: clearly defined boundaries, rules adapted to local conditions, arrangements by which affected parties take part in changing the rules, monitoring, graduated sanctions, accessible conflict-resolution mechanisms, minimal recognition of the right to organize, and nested

enterprises for larger systems. These principles map suggestively onto the five layers of Section 5: boundaries and authorization, collective choice and policy, monitoring and accountability, sanctions and revocation, and conflict resolution and dispute resolution. The mapping is an analogy that guided the decomposition. It is not a derivation, and software agents differ from human communities in ways that may invalidate parts of it, for example in the speed of action, the cost of identity creation, and the absence of social sanction.

North (1990) defines institutions as the humanly devised constraints that structure interaction and argues that they shape economic performance through their effect on transaction and information costs. Williamson (1985) analyzes governance structures such as markets, hierarchies, and hybrids as responses to transaction costs, and argues that the choice among them depends on asset specificity, uncertainty, and frequency of transactions. Brousseau and Glachant (2008) provide a guidebook to this new institutional economics. Read together, these works support a hypothesis: the choice between central orchestration and decentralized governance among agents is an institutional choice with costs on both sides, not a purely technical one.

Lessig (1999) argues that software architecture regulates behaviour, alongside law, social norms, and markets, so that 'code' can function as a form of law. If so, an agent ecosystem's merge rules, admission predicates, and key registries are already regulation, and the question is whether they are designed as such or arise accidentally. DeNardis (2014) argues that control over internet infrastructure and protocols is a locus of governance conflicts, which supports the point that protocol-level choices carry governance consequences.

Weitzner et al. (2008) argue that information systems should complement preventive access control with *accountability*: making it possible, after the fact, to determine whether a use of information complied with the rules that applied. This is directly relevant to the accountability layer proposed in Section 5, which we frame as record-keeping that supports after-the-fact determination, and not as a guarantee of prevention. Schneier (2012) argues that societies sustain cooperation through several pressures, namely moral, reputational, institutional, and security systems, and that security mechanisms are one pressure among several. The relevance here is a caution: technical authorization is one pressure, and an agent ecosystem lacks several others, which may make the technical pressure carry more load than it can bear.

Within computer science, the formal ingredients that a governance design would reuse are well known: digital signatures (Bernstein et al., 2012), hash-linked records (Merkle, 1988), Byzantine fault tolerance (Lamport et al., 1982; Castro and Liskov, 1999), the Sybil problem for open identities (Douceur, 2002), and the limits and opportunities of eventual consistency (Bailis and Ghodsi, 2013). Structured replicated data, such as JSON-like documents (Kleppmann and Beresford, 2017), is relevant to representing policies as mergeable artifacts. Access-control models such as role-based, attribute-based, and capability-based schemes are also relevant; we mention them as general categories and do not review them in this paper, which is a limitation (Section 7.4).

2.3 Governance vs. Verification (Orthogonality)

A key claim of the candidate is that governance is orthogonal to verification: that the question 'may this agent do this?' is distinct from 'does this action satisfy its specification?'. We define the two questions precisely in Section 5.4 and test the claim as F2. Here we note why the claim is not trivial. An authorization decision can itself be phrased as a proposition (agent X holds a valid chain of delegations for scope S) and, in principle, checked by a proof tool. If every authorization decision could be so encoded without loss, governance would be a special case of verification. Whether that is so depends on where the trust anchors come from, which is the crux of F2.

2.4 Governance vs. Orchestration (Boundary)

Orchestration decides *what agents do next*: it decomposes work, assigns tasks, schedules, and routes dependencies. Governance decides *what agents may do*, regardless of what they are assigned. The boundary is clear in principle: an orchestrator may assign a task that the assignee is not authorized to perform, and an authorized agent may act without being assigned. In a centrally orchestrated deployment, however, the orchestrator is typically also the policy decision point, so the two collapse in practice. The investigation therefore asks whether the distinction survives in a decentralized setting (F3).

2.5 The Falsification-First Methodology

The methodology is an application of the general principle that a hypothesis is worth taking seriously to the extent that it makes claims that could be shown false. We use the principle to structure the investigation, not to make claims about the philosophy of science. Concretely, the failure conditions of Section 3 are stated first, and the evidence sought is evidence that *would show failure*. Evidence that merely agrees with the candidate is treated as weak, because a design that is described by its proponent can always be made to agree with itself.

2.6 Research Gap

The literature supplies (i) institutional analyses of governance among humans, (ii) technical building blocks for authentication, signing, replication, and fault tolerance, and (iii) the Aether preprints, which supply architecture for four other concerns. What it does not supply, as far as this review can establish, is an analysis of whether authority, policy, accountability, revocation, and dispute resolution form a distinct concern for *decentralized, autonomous, code-modifying agents*, and of whether that concern can be handled without reintroducing a central authority. We state this as a gap in the material reviewed here, not as a claim about all existing work.

Limitations of this section. The review is selective and non-systematic. The institutional works are cited for framing and each analogy is a hypothesis. Technical access-control literature was not reviewed. Statements about the Aether records follow the author's own preprints; no independent reader has verified them.

3. The Falsification-First Methodology

3.1 The Problem with Prior Assumptions

A typical architecture paper begins from a conviction that a component is needed and then specifies it. The risk is circular: the specification is judged against the author's own framing of the problem, and the framing was chosen to make the component look necessary. The Aether family has a further risk of its own. Each earlier work was motivated as the answer to a question that the preceding works left open, and the sequence of questions (store, integrate, prove, distribute) has a momentum. A fifth question, 'who is authorized?', fits the sequence naturally, and that fit is exactly what should make us cautious. (*Observation:* the fact that a candidate fits a narrative is not evidence that it fills a gap.)

Three prior assumptions in particular deserve explicit challenge. **First**, that the existing dimensions have not already addressed authority; Section 2.1 observes that fragments exist in AetherMesh and AetherProof. **Second**, that authority questions are distinct from verification questions; Section 2.3 explains why this is not obvious. **Third**, that governance can be decentralized; revocation in an eventually consistent system is intrinsically delayed, and a design that ignores this would be unsound.

3.2 The Five Failure Conditions (F1–F5)

Each failure condition is stated as a condition whose *satisfaction* would show that the candidate fails. For each we give the claim under test, what would falsify it, and what kind of evidence is admissible. Appendix B gives full detail.

ID	Claim under test	Falsified if	Admissible evidence
F1 Non-redundancy	Some authorization requirement is not met by the existing dimensions' specified mechanisms plus a bounded, localized extension of one of them	Every requirement in the list R1–R6 below is met by existing mechanisms or by a bounded extension of a single existing dimension	Mapping of requirements to existing specifications; extension-size analysis
F2 Orthogonality	Authorization decisions, and the authority from which they derive, cannot be reduced to proof obligations in AetherProof without loss	Every authorization decision and every trust anchor can be encoded as a verified proposition with no residual unverifiable assumption about who may declare it	Encoding attempt; analysis of residual assumptions
F3 Boundary	Governance is distinct from orchestration in a decentralized setting	Governance reduces to task assignment plus conventional access control located in an orchestrator	Scenario analysis with and without an orchestrator
F4 Decentralizability	Governance can be specified without a trusted central authority and without breaking the determinism that AetherMesh requires	Any design that preserves the required properties needs a central authority or strong consistency for its core operations	Formal argument; counterexample search
F5 Observable benefit	There exist scenario classes in which the family with governance outperforms the family without it on a pre-specified metric	No scenario class can be constructed in which the best existing baseline fails and governance succeeds	Pre-registered simulation or deployment study (not yet done)

The requirement list used by F1. We define six candidate requirements; this list is itself a hypothesis and is revisited in Appendix B.

ID	Requirement (definition)
R1	Scoped, attenuating delegation: an agent can grant a subset of its authority to another, and no agent can grant more than it holds.
R2	Authorized policy change: the rules that decide authorization can themselves be changed only by agents authorized to change them.
R3	Revocation with dependents: withdrawing a grant also withdraws grants derived from it.
R4	Attribution: each admitted action can be traced to the chain of agents responsible for it.
R5	Dispute handling: conflicting authority claims are resolved by a deterministic rule or an explicit escalation.
R6	Partition tolerance: authority checks can be made at a node that is temporarily disconnected, with a stated safety consequence.

3.3 The Decision Rule

The decision rule maps outcomes of the five conditions to what the author will conclude. It is stated before any outcome is known, so that it cannot be adjusted to fit one. We use the terms *single*, *dual*, and *triple* project to mean, in our reading, a project structure that develops one, two, or all three of the candidate capabilities of Section 4 as new work; the final structure is not decided here.

Outcome	Conclusion	Consequence for project structure
F1 falsified	Governance requirements are already met or are a bounded extension of one existing dimension	No standalone governance project; fold the extension into that dimension

Outcome	Conclusion	Consequence for project structure
F2 falsified	Governance is a special case of verification	Fold into AetherProof; no standalone governance project
F3 falsified	Governance is orchestration plus access control	Governance merges with the orchestration candidate; at most one project covers both
F4 falsified	Governance cannot be decentralized	Incompatible with the family's premises; do not pursue as a dimension of this family
F5 falsified	No observable benefit can be shown	Do not pursue; retain analysis as a negative result
None falsified, F5 untested (current state)	Candidate not falsified; not confirmed	Undetermined; proceed to comparison and to a pre-registered evaluation
None falsified, F5 supported by evidence	Candidate supported in the scenarios tested	Standalone project becomes defensible; structure decided by comparison with the other two candidates

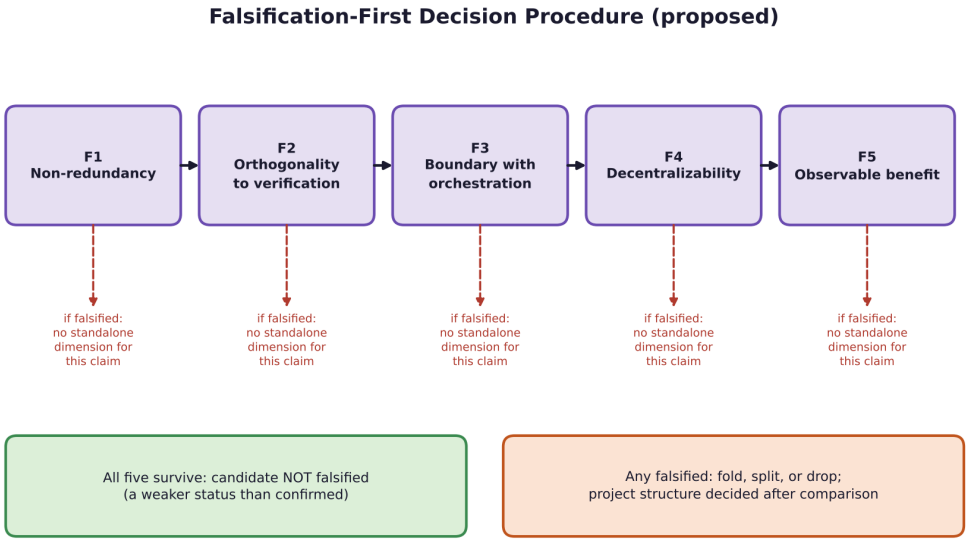


Figure 2 (Fig7_Falsification_Procedure.png; supplementary). The decision procedure. Each condition is a possible point of failure; surviving all five yields 'not falsified', a weaker status than 'confirmed'.

Two features of the rule deserve comment. First, the status 'not falsified' is never promoted to 'confirmed' on the basis of argument. Second, a falsified condition does not discard the analysis: it redirects the material to the dimension into which it folds.

3.4 Why This Methodology Matters

The methodology matters here for three reasons. It protects against the narrative momentum noted above. It makes the paper's eventual conclusion informative regardless of its direction: a result that governance folds into AetherMesh would be as useful to the family as one that it stands alone. And it allocates effort honestly: the cheapest ways to be wrong (redundancy, reducibility) are examined first, the expensive ones (evaluation) last.

The methodology also has costs and limits. Failure conditions are chosen by the same author who proposed the candidate, so they may be too easy to survive; we mitigate this by stating, for each, the specific evidence that would falsify it and by naming residual weaknesses. An independent reviewer would be a stronger check than any self-imposed criterion, and we invite one (Section 8.4).

Limitations of this section. The five conditions are not claimed to be exhaustive or optimal. They are one reasonable set, chosen before the assessment of Section 5. The requirement list R1–R6 is a hypothesis; a

different list could change F1's outcome.

4. The Three Candidate Capabilities

Governance is not the only possible fifth dimension. Before examining it in depth, we describe three candidate capabilities that could plausibly fill a gap beside the four existing dimensions, so that the choice to investigate governance is itself exposed to comparison. The descriptions are definitions of candidates, not designs; the names (AetherOrchestrate, AetherGovern, AetherRuntime) are working labels.

4.1 Orchestration (AetherOrchestrate)

AetherOrchestrate (candidate) *(definition).* A capability that decomposes goals into tasks, assigns tasks to agents, schedules and routes work, and tracks dependencies among tasks, with the aim of coordinating what agents do next.

Gap addressed (hypothesis). The four existing dimensions say how changes are stored, integrated, proven, and synchronized, but not how work is divided among agents so that they do not duplicate or contradict one another. **Decentralization question.** Orchestration is conventionally central; a decentralized version would need a task-allocation mechanism that is itself replicated, which raises questions about agreement on assignments. **Principal risks.** The central version reintroduces the single point of failure that AetherMesh removes; the decentralized version may require consensus that gossip alone does not supply, and so may fall under the impossibility results for asynchronous agreement. **Relation to governance.** Assignment implies a kind of authority; whether authority can be derived from assignment alone is part of F3.

4.2 Governance (AetherGovern)

AetherGovern (candidate) *(definition).* A capability that determines which agents are authorized to perform which actions, under which policies, with what accountability, with what means of revocation, and with what means of resolving conflicting claims.

Gap addressed (hypothesis). Authority is implicit in the existing dimensions: a signature identifies an author, and a registry lists keys, but the grant, scope, delegation, withdrawal, and dispute of authority are not specified. **Decentralization question.** Revocation and policy change must propagate through an eventually consistent substrate, which creates exposure windows (Section 5.3). **Principal risks.** Trust anchors may reintroduce central control; the layer may be mere access control under a new name; benefits may be unmeasurable without a realistic threat model.

4.3 Runtime Assurance (AetherRuntime)

AetherRuntime (candidate) *(definition).* A capability that observes agents and their actions while they execute, enforces resource limits and behavioural constraints, detects anomalies, and can halt or isolate an agent.

Gap addressed (hypothesis). AetherFlow verifies after merge and AetherProof verifies against specifications, but neither monitors what an agent is doing while it runs, for example consuming resources or issuing unusual sequences of operations. **Decentralization question.** Monitoring can be local to each node, but a decision to isolate an agent must be communicated and agreed on, which touches governance. **Principal risks.** Runtime monitors can be evaded by adaptive agents; detecting anomalies without false positives is a hard open problem that this paper does not address; the capability may overlap with operational verification in AetherFlow.

4.4 Comparison of the Three Candidates

Criterion	Orchestration	Governance	Runtime assurance
Central question	What should agents do next?	Who may act, under which policy?	Are agents behaving acceptably while running?
Gap hypothesized	Division of work	Authority, accountability, revocation	Behaviour during execution
Closest existing dimension	AetherFlow (integration)	AetherMesh (key registry), AetherProof (review roles)	AetherFlow (operational verification)
Redundancy risk	Medium: may be an application of integration	Medium: fragments exist (Observations 1 and 3)	High: overlaps operational verification
Decentralization difficulty	High: agreement on assignment	Medium–high: revocation lag, trust anchors	Medium: local monitoring is easy, collective action is not
Interaction with the others	Needs authority to assign	Constrains what orchestration may assign	Needs authority to halt an agent
Evidence status	None	None (argument only in this paper)	None
Assessed against F1–F5 in this paper	No	Yes, provisionally	No

Table 4.1. Comparison of candidate capabilities. Entries are the author's qualitative judgements, not measurements.

The table shows an interdependence that argues against treating the candidates as independent: orchestration and runtime assurance both need an answer to 'who may assign?' and 'who may halt?', which is a governance question. This is an observation about dependency, not evidence that governance is the correct choice, because dependencies could equally be satisfied by access-control features embedded in each of the other two.

4.5 Which Capability Is the Strongest Candidate?

This paper does not rank the three candidates. It examines governance in depth because the author's preliminary reading is that governance has the largest number of fragments already present in the existing records (Observations 1–3), which makes the redundancy question most testable. A candidate whose redundancy can be tested cheaply is a good first subject for a falsification-first investigation, whether or not it turns out to be the best candidate.

> **Candidate statement** *(hypothesis).* AetherGovern is a candidate for addressing the authorization gap, provided it passes the falsification criteria (F1–F5).

No claim is made that the other two candidates are weaker. Their assessment against analogous failure conditions is future work (Section 8.4), and the final project structure will be determined after that comparison.

> **Limitations of this section.** The descriptions of orchestration and runtime assurance are brief and were not investigated to the depth given to governance, which biases the comparison. Table 4.1 is qualitative.

5. AetherGovern: Architecture

This section specifies the hypothesized architecture. All of it is hypothesis: a specification-level description of what a governance dimension might contain, offered so that the failure conditions can be applied to something concrete. Nothing here has been implemented.

5.1 The Five Governance Layers

We decompose governance into five layers, each addressing one question. The decomposition is guided by the design principles that Ostrom (1990) identifies for long-enduring commons institutions (Section 2.2); the mapping is an analogy. The ordering in Figure 3 reflects conceptual dependency, in that each layer presupposes the ones below it, and not an enforcement hierarchy.



Figure 3 (Fig2_Governance_Layers.png). The five layers of AetherGovern (hypothesis). Ordering reflects conceptual dependency, not an enforcement hierarchy.

Layer	Question	Objects	Operations	Design requirement (hypothesis)
1 Authorization	Who may act?	Agent identity; action; resource; scope; delegation certificate	grant, present, verify	Every admitted action carries a verifiable chain of authority for its scope
2 Policy	Under which rules?	Policy document; policy version; approval record	propose, approve, enforce, supersede	Policy changes are themselves actions subject to authorization
3 Accountability	Who is responsible?	Action record; chain of authority; signature; hash link	record, link, query	Each admitted action can be attributed to a chain of agents after the fact
4 Revocation	How is authority withdrawn?	Revocation record; revocation frontier; epoch	revoke, propagate, cascade	Revocation is monotone and dependent grants lapse with their parents
5 Dispute resolution	How are conflicts resolved?	Conflicting claims; precedence rule; escalation record	detect, apply rule, escalate	Conflicts are resolved by a deterministic rule or an explicit escalation

Layer 1, Authorization. The authorization layer decides whether an agent may perform an action on a resource (commit, merge, delete, deploy, rotate keys). In the proposed model, authority is represented by signed *delegation certificates* (Section 5.3) and an action is authorized if it presents a chain of certificates that verifies. Signature verification would use a standard scheme (Bernstein et al., 2012). **Layer 2, Policy.** Policies are the rules that govern authorization: who can grant authority, under which conditions, for how long. A policy may be stored as a structured, mergeable document (cf. Kleppmann and Beresford, 2017), but policy merging raises its own conflicts, treated in Layer 5. **Layer 3, Accountability.** Following the argument of Weitzner et al. (2008) that after-the-fact accountability complements prevention, each admitted action is recorded together with its chain of authority in a hash-linked log (Merkle, 1988) so that

responsibility can be determined later. **Layer 4, Revocation.** Authority may be withdrawn when an agent is compromised, misbehaves, or leaves. Section 5.3 analyzes the consequences of doing so in an eventually consistent substrate. **Layer 5, Dispute resolution.** Conflicts between authority claims, for example two certificates that grant overlapping scopes under incompatible conditions, or a revocation concurrent with a re-delegation, are resolved by a deterministic precedence rule where one exists and by explicit escalation otherwise.

Limitations of this section. The decomposition into exactly five layers is a choice, not a derivation. Other decompositions are possible, and the Ostrom mapping may mislead where software agents differ from human communities.

5.2 The Policy Lifecycle

A policy in AetherGovern passes through four stages, shown in Figure 4: **creation** (an agent proposes a policy, for example that only agents with role X may modify directory Y), **approval** (authorized agents approve it, recording their signatures), **enforcement** (every action is checked against the policy before it is admitted), and **revocation** (the policy is withdrawn when no longer needed or when found flawed). Revocation returns the system to a state in which a new policy may be proposed.

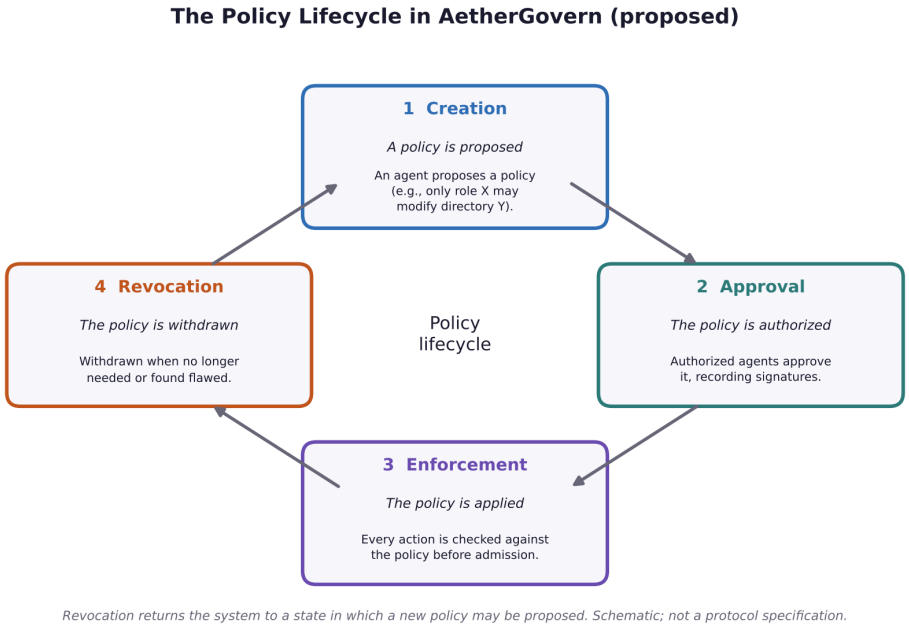


Figure 4 (Fig3_Policy_Lifecycle.png). The policy lifecycle in AetherGovern (hypothesis): creation, approval, enforcement, revocation. Schematic; not a protocol specification.

Three design questions arise. First, *who may approve a policy?* The answer cannot be a policy, on pain of regress; it must be a root of authority fixed outside the policy system (Section 5.3). Second, *what is the effective date of a policy?* In an eventually consistent system, different nodes learn of a policy at different times; the proposal here is that every action cites the policy version it was checked against, so that verdicts remain deterministic (Proposition 1). Third, *can policies conflict?* Concurrent approvals of incompatible policies are possible; Layer 5 supplies a precedence rule, for example that the more restrictive policy prevails until escalation resolves the conflict. This 'restrictive prevails' rule is a conservative default and is a hypothesis, because it favours safety over availability and can block legitimate work.

Limitations of this section. The lifecycle is schematic. Approval thresholds, expiry, versioning, and the handling of policies that apply retroactively are unspecified (Q4, Q5).

5.3 Authority Delegation

Delegation is the mechanism by which authority held by one agent is conferred, in whole or in part, on another. Figure 5 shows a chain in which Agent A, the root of authority, delegates to Agent B, who delegates to Agent C.

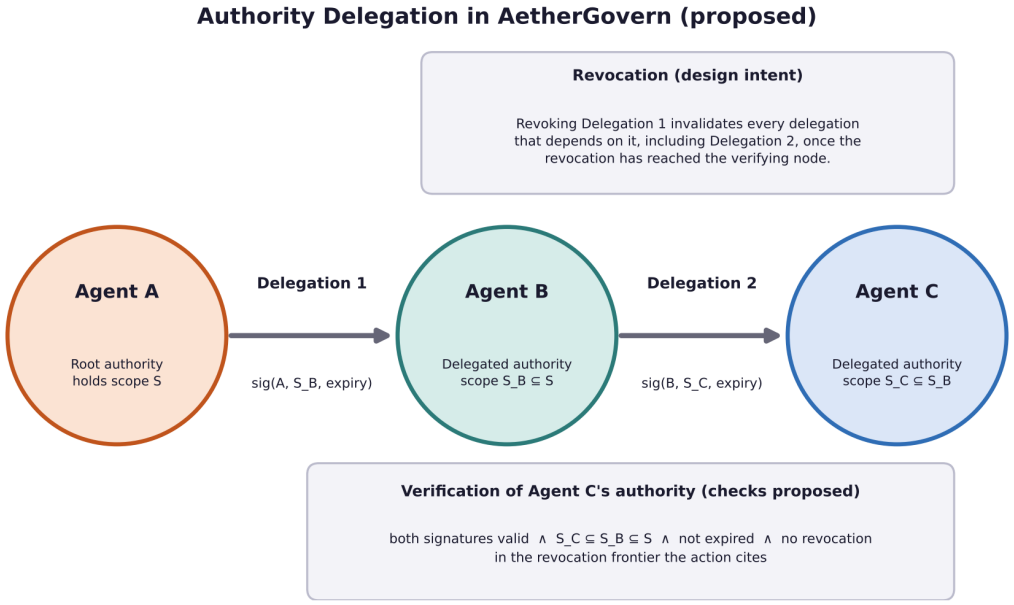


Figure 5 (Fig4_Authority_Delegation.png). Authority delegation in AetherGovern (hypothesis). Each delegation is a signed certificate; scopes may only shrink along the chain. The verification conditions shown extend those in the originally supplied figure by scope attenuation, expiry, and a revocation check.

Delegation certificate *(definition).* A record (issuer, subject, scope, constraints, expiry, parent reference, signature) in which the issuer authorizes the subject for the stated scope. A *chain* is a sequence of certificates in which each certificate's issuer is the previous certificate's subject, and the first certificate's issuer is a root of authority.

Chain validity at frontier F *(definition).* A chain is valid at revocation frontier F if (a) every signature verifies; (b) scope attenuates: $\text{scope}(c_{i+1}) \subseteq \text{scope}(c_i)$ for each consecutive pair; (c) no certificate has expired at the action's logical time; (d) no certificate in the chain appears in the revocation set recorded in F; and (e) the first certificate's issuer is a recognised root.

Condition (b) is the technical content of the attenuation property (R1): an agent cannot grant more than it holds. The originally supplied description of verification, 'Signature(A, scope_B) ∧ Signature(B, scope_C)', omits (b) through (e); without them, a delegation by B of a scope larger than B holds would verify. This is an observation about the supplied sketch and is corrected here. Verification cost is linear in chain length, so a depth limit is a natural design requirement (Q17).

5.3.1 Causal-authority admission and the determinism requirement

AetherMesh's convergence argument (its Theorem 1) requires that each admission predicate be a deterministic function of a delta and its causal closure (its assumption A6). Authority checks threaten this requirement, because whether a certificate is valid depends on which revocations a node has *already seen*, and that is local, time-varying state. Two honest nodes could therefore give different verdicts on the same action. We propose a remedy and state it as a proposition.

Causal-authority admission *(definition).* Every action record includes a reference to a *revocation frontier* F, a set of revocation records (identified by hash) that the actor had seen when it acted, and to the policy version it relied on. A node evaluates chain validity at the frontier F named by the action, and

not at its own current revocation set. Revocation records and policy records are deltas, and the frontier and policy references are part of the action's causal closure.

Proposition 1 (determinism of causal-authority admission) *(stated with proof sketch; not machine-checked).* If the admission predicate for an action consists of signature, attenuation, expiry, root recognition, and revocation checks evaluated at the revocation frontier cited by the action, and the cited frontier and policy version are in the action's causal closure, then the verdict is a deterministic function of the action and its causal closure, and the predicate satisfies assumption A6 of AetherMesh.

Proof sketch. Each check is a pure function of data contained in the action, the certificates in its chain, the revocation records named by the frontier, and the policy version it cites. Signature verification and hash comparison are deterministic given the key material; attenuation is a set-inclusion test; expiry compares the action's logical time to the certificate's expiry, both contained in the data; the revocation check tests membership in a set fixed by the frontier. None of these reads local state beyond the closure. Hence two honest nodes that hold the closure compute the same verdict. If a cited item is not yet present, the action is deferred, as AetherMesh already does for missing parents. ∎ (sketch)

What the sketch does not show. It assumes that the root-of-authority set is common knowledge among honest nodes and is itself part of the closure; changing roots is outside the proposition (Q10). It guarantees determinism, not security: an action that cites a stale frontier is admitted at every honest node even if a later revocation exists, which is the subject of the next proposition. It does not address clock manipulation: expiry compares logical times that an actor reports, so an actor could understate its time (Q12).

5.3.2 Revocation and its exposure window

Revocation set *(definition).* The set of revocation records known at a node. Adding a revocation to the set can only invalidate certificates, never validate them.

Proposition 2 (monotonicity of revocation) *(stated with proof sketch; not machine-checked).* Let valid(c, F) denote 'chain c is valid at frontier F'. If $F \subseteq F'$ then valid(c, F′) ⇒ valid(c, F). Equivalently, validity is antitone in the revocation frontier, and the revocation set can be maintained as a grow-only set whose join is union.

Proof sketch. Of the five validity conditions, only (d) depends on F, and it is a non-membership test in the set named by F. If $F \subseteq F'$, then any record in F is in F′, so non-membership in F′ implies non-membership in F. The revocation set is thus a grow-only set, which is a join-semilattice under union (Shapiro et al., 2011), and can be replicated with delta-state techniques (Almeida et al., 2018). ∎ (sketch)

Proposition 2 is good news for decentralization, because revocation requires no coordination to *record*. The cost is in *timeliness*. Because an action by a compromised agent may be issued before the issuing node hears of the revocation, there is an interval in which actions that cite the revoked authority are legitimately admitted by honest nodes (Figure 6).

DOI: https://doi.org/10.5281/zenodo.24054524

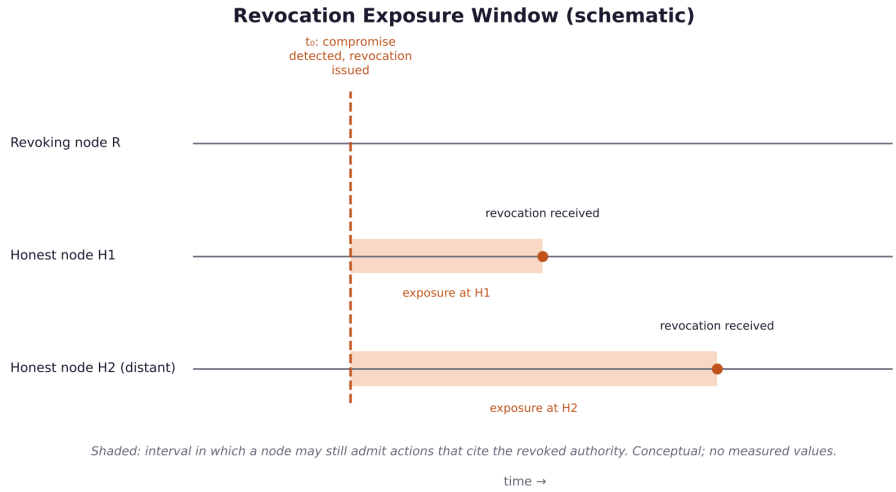


Figure 6 (Fig6_Revocation_Window.png; supplementary). Schematic of the exposure window: the interval between issuing a revocation and its receipt at a node, during which that node may still admit actions citing the revoked authority. Conceptual; no measured values.

Observation (exposure window) *(observation).* In any system in which nodes act on locally held information and revocations propagate by message passing, an action issued by a node that has not yet received a revocation is concurrent with the revocation, and a rule that rejects every concurrent action is not implementable without coordination among all nodes. Hence, absent such coordination, the exposure window cannot be eliminated by design.

Argument. Concurrency is a relation between events, not a property that a protocol can remove: if the action and the revocation are causally unordered, neither node's decision can depend on the other event. Preventing the action would require the acting node to consult every other node before acting, which is coordination. This is the familiar trade-off between availability and strong agreement in the presence of partitions, and is not specific to this proposal.

Hypothesis H1 (window size) *(hypothesis; not established).* If revocations are disseminated by epidemic gossip, the expected exposure window at a node is of the order of the dissemination time of a delta, which AetherMesh's heuristic estimate places at a small multiple of ln n gossip rounds. Under the constant-hazard model of AetherMesh's Theorem 2, the probability that a node has not yet received a revocation after t steps decays as $e^{-\beta t}$.

This hypothesis is a heuristic transfer of an unverified result: AetherMesh's own latency figures are simulation-reported, and its analysis does not reproduce them across network sizes. We state H1 so that it can be tested (Section 7), not because we consider it established. Two policy choices follow, and we describe them without recommending one.

Choice	Rule	Property	Cost
Causal validity	An action is valid if its cited authority was valid at its cited frontier	History is monotone; AetherMesh convergence is preserved (Proposition 1)	Actions in the exposure window remain valid; harm during the window must be remediated by other layers
Eventual validity	An action is invalid if any revocation, seen later, covers its authority	Stronger security; closes the window retroactively	Non-monotone: previously admitted actions are later quarantined, breaking AetherMesh's inflationary state and requiring rollback or quarantine semantics

Table 5.1. Two ways of treating actions concurrent with a revocation. The author has not decided between them; a hybrid with quarantine is also conceivable (Q9).

Trust anchors. Chain validity condition (e) requires that the first certificate's issuer be a recognised root. This is the point at which any such design relies on something outside itself. A single root is a central authority in the trust sense; several roots with threshold agreement reduce but do not remove the dependence, and agreement among roots falls under classical Byzantine bounds (Lamport et al., 1982; Castro and Liskov, 1999). In an open setting, identity creation is cheap and Sybil identities (Douceur, 2002) can undermine any scheme that counts agents. We regard the root problem as the principal residual weakness of F4 (Section 5.8).

Limitations of this section. Propositions 1 and 2 are proof sketches. Proposition 1 secures determinism, not safety. The observation on exposure windows is an argument from concurrency, not a formal impossibility theorem. H1 depends on AetherMesh results that are themselves unverified.

5.4 Governance vs. Verification

Governance and verification answer different questions. Governance asks whether an agent *may* perform an action; verification asks whether the action, once performed, *satisfies* its specification. Figure 7 contrasts them.

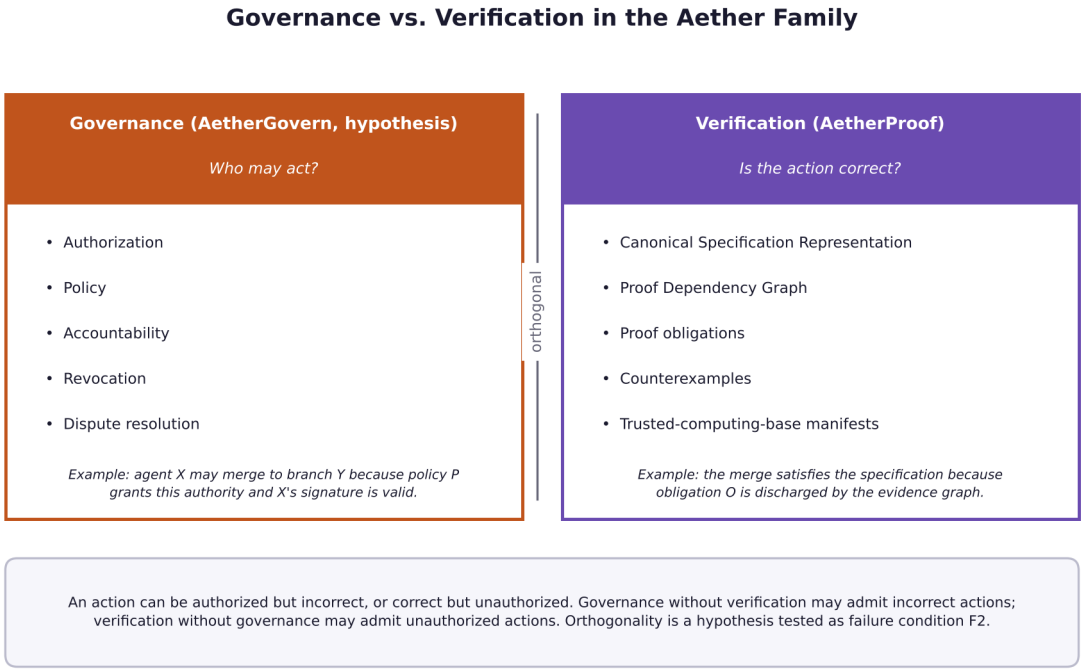


Figure 7 (Fig5_Governance_vs_Verification.png). Governance versus verification in the Aether family. Orthogonality is a hypothesis tested as F2; the figure was redrawn for this record.

	Correct (satisfies specification)	Incorrect (violates specification)
Authorized	Admit. The normal case.	Admit under governance alone; verification must catch it. Governance without verification admits incorrect actions.
Unauthorized	Verification alone would admit it; governance must reject it. Verification without governance admits unauthorized actions.	Reject by either; the redundancy is harmless.

Table 5.2. The four combinations of authorization and correctness. The off-diagonal cells are what make the two concerns complementary, if the orthogonality hypothesis holds.

Where the two meet. The concerns are not independent in every respect. Specifications are themselves artifacts that someone must be authorized to change; in AetherProof the human review protocol governs such changes in the verification domain. Conversely, a policy is a specification of authorization and could

in principle be checked by a verifier for consistency (for example, that no two policies conflict). We therefore state orthogonality at the level of the *question asked*, not at the level of mechanism: the mechanisms may share tools, but the trust anchor for 'who may declare a specification' cannot be supplied by verification, because verification takes specifications as input. That last argument is the core of the case against F2's falsification and is a hypothesis.

Limitations of this section. The orthogonality argument is conceptual. A formal encoding of delegation chains in a proof language was not attempted, which is the experiment that could falsify F2 (Q7).

5.5 Inter-Dimensional Harmonization

A governance dimension must coexist with the other four without making any of them depend on it in a way that breaks their own claims. We state four harmonization principles as design requirements.

1. **Order of checks.** Authorization checks are cheap (signature and set operations) and verification is potentially expensive; evaluating authorization first lets unauthorized actions be rejected before any proof effort is spent. This is a cost argument, not a safety argument.
2. **Determinism.** Every governance check that participates in admission must satisfy Proposition 1, so that AetherMesh's convergence argument is preserved. Checks that cannot (for instance rate limits) are delivery policies that delay but do not reject.
3. **Monotone records.** Governance records (certificates, revocations, policy approvals) are append-only deltas, so that AetherGit-style logs and AetherMesh-style replication can carry them without new mechanisms.
4. **Separation of authority and evidence.** Authority records say who may act; evidence records (proofs, test results) say what is true. Neither is derived from the other.

5.6 Design Principles

ID	Principle (design requirement)	Rationale
P1	Least authority: certificates carry the narrowest scope adequate for the task	Limits damage from compromise
P2	Attenuation only: no agent grants more than it holds	Prevents privilege escalation along chains
P3	Time-bounded grants: every certificate has an expiry	Bounds the consequence of missed revocation
P4	Monotone revocation: revocations are only added	Allows coordination-free replication (Proposition 2)
P5	Causal citation: actions cite their frontier and policy version	Preserves determinism (Proposition 1)
P6	Explicit trust anchors: roots are declared, not inferred	Makes the residual central dependency visible
P7	Accountability by record: attribution is by after-the-fact query of signed records	Matches the accountability argument of Weitzner et al. (2008)
P8	Escalate, do not guess: unresolved conflicts go to an explicit escalation	Avoids silent arbitrary resolution

5.7 The Five Functional Dimensions

Figure 1 places AetherGovern beside the four existing dimensions. **The fifth dimension (AetherGovern) is a hypothesis under investigation, not a confirmed dimension.** It is drawn with a dashed outline for that reason. The dimensions are functional, not hierarchical: AetherGovern would not sit above or below the others but would constrain what each of them may admit.

Dimension	What it admits or produces	What governance adds (hypothesis)
AetherGit	Changesets in an append-only log	Which agent may append, to which branch or path

Dimension	What it admits or produces	What governance adds (hypothesis)
AetherFlow	Integrated changes after verification	Which agent may trigger remediation or override a verdict
AetherProof	Specifications, obligations, evidence	Which agent may change a specification or accept evidence
AetherMesh	Replicated deltas	Which keys are recognised, and how recognition is withdrawn

5.8 Provisional Assessment Against F1–F4

We now apply the failure conditions to the candidate as specified, using argument alone. The assessments are provisional: they record whether we found a way to falsify each condition, not whether none exists.

Condition	Attempted falsification	Result	Residual weakness
F1 Non-redundancy	Map R1–R6 to existing mechanisms. R1 (attenuating delegation), R3 (revocation with dependents) and R5 (dispute handling) are not specified in any of the four records. R2 and R4 are partly present (review roles; signed attribution). R6 is partly present (local registry).	Not falsified for R1, R3, R5	A bounded extension of AetherMesh's key-registry and membership mechanism might absorb R1 and R3; whether that extension is 'bounded' is a judgement (Q1)
F2 Orthogonality	Encode authorization as a proposition in AetherProof. Possible for chain validity given anchors. Not possible for the anchors themselves (who may declare a root, a policy, or a specification).	Not falsified, narrowly	The argument rests on a distinction between decision and anchor that a critic may reject; no formal encoding attempted (Q7)
F3 Boundary	Separate governance from orchestration with no orchestrator present. Authority checks still needed at every admitting node; task assignment absent.	Not falsified in decentralized setting	In a centralized deployment the orchestrator can serve as the policy point, so F3 may fail there (Q8)
F4 Decentralizability	Design without a central authority: Proposition 1 preserves determinism; Proposition 2 permits coordination-free revocation records. Search for a requirement needing strong consistency: revocation timeliness.	Not falsified, with explicit residuals	Exposure window is irreducible; roots are a central dependency in trust terms (Q9, Q10)
F5 Observable benefit	Not attempted: requires an experiment.	Untested	Everything depends on it (Section 7)

The overall status is therefore *not falsified on F1–F4 by argument, with residuals; F5 untested*. This status does not justify a standalone project. It justifies the evaluation design of Section 7 and the comparison with the other candidates.

> **Limitations of this section.** The assessment is made by the same author who proposed the candidate, using argument alone. Independent review and a formal encoding attempt would be stronger tests.

6. AetherGovern: Interfaces

An interface is a statement of what AetherGovern would require from, and provide to, another dimension. We specify interfaces so that the coupling can be examined: a governance dimension that required deep changes to every other dimension would fail F1 in the opposite direction, by being too entangled to stand alone. All interface descriptions are hypotheses; the abstract interfaces of the other dimensions are as described in their preprints.

6.1 Interface with AetherGit

Requires. A storage schema in which a changeset can carry an authority chain reference and a policy-version reference, and in which governance records (certificates, revocations, approvals) can be stored as ordinary append-only records. **Provides.** A verdict on whether the author of a changeset was authorized for the paths it touches. **Risk.** If the storage schema is fixed, adding fields changes identifiers; the mapping is unspecified (Q14).

6.2 Interface with AetherFlow

Requires. An admission hook in the integration protocol where an authorization verdict can be obtained before verification is scheduled. **Provides.** Authorization verdicts, and a rule for who may initiate or approve remediation actions. **Risk.** Remediation changes are themselves actions; if automated remediation is performed by an agent, it needs authority like any other agent (Q15).

6.3 Interface with AetherProof

Requires. A designation of which specification changes and which evidence acceptances are governed actions. **Provides.** A statement of who may change a specification, and policy documents that can be treated as specifications and checked for consistency. **Risk.** Circularity: if policies are verified by AetherProof and AetherProof's own decisions are governed by those policies, the dependency must be grounded in a root (Q16).

6.4 Interface with AetherMesh

Requires. That governance records replicate as deltas, and that the admission pipeline accept an additional authorization predicate that satisfies the determinism requirement (assumption A6). **Provides.** A specified registry mechanism with delegation, rotation, and revocation, addressing what AetherMesh leaves unspecified (its T6 and Q31). **Risk.** This is the closest coupling and the one most likely to be judged a bounded extension of AetherMesh (F1). If a reviewer concludes that governance is simply AetherMesh's membership and key management done properly, F1 is falsified for that part.

6.5 Interface Summary

Dimension	Requires from it	Provides to it	Coupling
AetherGit	Fields for authority and policy references; append-only governance records	Authorship authorization verdict	Low–medium
AetherFlow	Admission hook before verification; role for remediation	Authorization verdicts; remediation authority	Medium
AetherProof	Designation of governed specification changes	Authority for specification change; checkable policies	Medium
AetherMesh	Delta replication of governance records; determinism-compatible predicate	Registry with delegation, rotation, revocation	High

DOI: https://doi.org/10.5281/zenodo.24054524

6.6 Minimal vs. Full Interface

A **minimal interface** consists of three artifacts and one rule: a delegation certificate record, a revocation record, an authority-frontier reference carried in each action, and the requirement that the authorization predicate satisfy Proposition 1. This is the smallest interface that would give the other dimensions scoped delegation and revocation, and its smallness is itself relevant to F1: if the minimal interface is all that is needed, the whole contribution might be an extension of AetherMesh. A **full interface** adds policy lifecycle messages, accountability queries, dispute and escalation messages, the governed-specification designation for AetherProof, and the remediation-authority rule for AetherFlow.

	Minimal	Full
Records	Delegation certificate; revocation record	Plus policy, approval, accountability, dispute and escalation records
Action header	Authority-frontier reference	Plus policy-version reference
Predicate	Chain validity at cited frontier	Plus policy enforcement and conflict rule
Layers covered	1 (Authorization), 4 (Revocation)	All five
F1 implication	Likely a bounded extension of AetherMesh	Harder to fold into a single dimension

The contrast sharpens the falsification question: the case for a standalone dimension rests on the layers beyond the minimal interface (policy, accountability, dispute resolution) being separable and valuable. That is a hypothesis for which this paper has no evidence.

Limitations of this section. Interfaces were specified at the level of requirements. No message formats, schemas, or sequence diagrams are given, and the coupling ratings are qualitative.

7. Evaluation

No evaluation has been performed. This section specifies how the candidate would be evaluated, in a form that fixes in advance what outcomes would count as failure. Because F5 (observable benefit) is the only failure condition that argument cannot settle, this design is the main deliverable of the section. Every metric, baseline, and threshold below is a proposal.

7.1 Evaluation Metrics

ID	Metric	Definition (proposed)	Direction
M1	Unauthorized-action admission rate	Fraction of actions by agents lacking valid authority that are admitted	Lower is better
M2	Exposure count	Number of actions citing revoked authority that are admitted after the revocation is issued	Lower is better
M3	Exposure duration	Time from revocation issue to its receipt at each honest node (median, 99th percentile)	Lower is better
M4	Attribution completeness	Fraction of admitted actions for which the responsible chain can be recovered from records	Higher is better
M5	Attribution accuracy	Fraction of recovered chains that match the ground-truth chain in the simulation	Higher is better
M6	False-rejection rate	Fraction of legitimate actions rejected by governance checks	Lower is better
M7	Dispute handling	Median time to resolve conflicting claims; fraction escalated	Lower is better (time)
M8	Admission overhead	Added admission latency, CPU, and storage per action relative to a baseline with no governance	Lower is better
M9	Convergence preservation	Count of runs in which honest nodes end with different admitted sets	Must be zero if Proposition 1 holds

M9 is a falsification check on Proposition 1: a single run in which honest nodes diverge after all messages are delivered would refute the proposition as stated, or reveal an implementation or modeling error. M3 is the direct test of Hypothesis H1.

7.2 Comparison with Baselines

A comparison is informative only if the baselines are the strongest reasonable alternatives that do not use the governance architecture. The risk of a weak baseline is acknowledged in Section 7.4.

ID	Baseline	Description	What it tests
B1	No authority checks beyond AetherMesh admission	Signature validity and a static key registry; no delegation, revocation, or policy	The value of any governance mechanism
B2	Static registry with periodic refresh	Registry updated by an out-of-band push at fixed intervals	Whether simple key rotation suffices (F1)
B3	Centralized access control	A single policy decision point consulted by all nodes; strongly consistent	Cost and benefit of decentralization versus a central authority (F4)
B4	Expiring bearer grants without revocation	Short-lived certificates; no revocation propagation	Whether expiry alone bounds the exposure window
B5	Minimal-interface governance	Certificates, revocations, frontier reference only (Section 6.6)	Whether layers beyond the minimal interface add value

B2 and B4 are the most important baselines for F1: if short-lived grants (B4) or periodic registry refresh (B2) achieve results comparable to full governance, the case for a separate dimension weakens materially. B3 is not a competitor in the family's terms, since it reintroduces a central authority, but it defines the performance and safety that decentralization gives up or retains.

7.3 Evaluation Methodology

Platform. A discrete simulation of a peer-to-peer network, extending the AetherMesh testbed described in its preprint, which is itself a simulation with a simplified network model. A deployed study is out of scope for the first stage.

Scenario classes. Six classes are proposed, each corresponding to a requirement in Section 3.2: S1 *key compromise* (a delegated agent's key is stolen and used; tests R3, M1–M3); S2 *over-reach* (an authorized agent acts outside its scope; tests R1, M1, M6); S3 *post-incident attribution* (a harmful action is admitted and the responsible chain must be recovered; tests R4, M4–M5); S4 *concurrent policy change* (two incompatible policies approved concurrently; tests R2, R5, M7); S5 *delegation abuse* (a chain that exceeds scope or depth; tests R1, M1); and S6 *partitioned operation* (revocation issued during a partition; tests R6, M2–M3). In addition, two *negative-control* classes are included in which governance should provide no benefit, for instance benign operation with no compromised agents, so that overhead (M8) and false rejections (M6) are measured where there is nothing to gain.

Adversary models. We separate three: an external attacker with a stolen key (S1), an insider that abuses legitimate authority (S2, S5), and a colluding group that manipulates policy approval (S4). A Sybil adversary (Douceur, 2002) is a fourth model deferred to later work (Q11).

Pre-registration. Before any run, the following would be recorded: the scenario parameters, the metric definitions, the seeds, the number of runs, the analysis, and the decision thresholds below. After data collection, thresholds are not changed. This is the experimental counterpart of the falsification-first principle.

Proposed decision thresholds. These values are proposals, fixed by the author for the purpose of the design, and may be revised *before* data collection by a reviewer; they are not results.

Outcome	Criterion (proposed)
F5 supported in a scenario class	Governance configuration reduces M1 or M2 relative to the best of B1, B2, B4, with a 95% bootstrap confidence interval for the difference that excludes zero over at least 30 independently seeded runs, while M6 does not increase and M8 added admission latency is at most 10% of the baseline
F5 falsified	No scenario class meets the previous criterion
Proposition 1 refuted	Any run with M9 > 0 after full delivery with the causal-authority predicate
H1 supported	Observed M3 distribution is consistent, in order of magnitude, with the gossip dissemination time measured for ordinary deltas in the same run; 'consistent' is defined by the analysis plan before data are seen
Minimal interface sufficient	B5 meets the F5 criterion in every class in which full governance does, in which case layers 2, 3 and 5 are not shown to add value

Ablation. The five layers would be enabled cumulatively (1; 1+4; 1+4+3; 1+4+3+2; all) so that the contribution of each can be seen. This matters for the minimal-versus-full question: the case for a standalone dimension is stronger if layers 2, 3 and 5 contribute measurably.

7.4 Threats to Validity

Type	Threat	Mitigation / status
Internal	The same author proposes the candidate, the failure conditions, the baselines, and the thresholds	Pre-registration; invite external review of conditions and thresholds before data
Internal	Weak or straw-man baselines make governance look better	Include B2, B4, B5, which are deliberately strong simple alternatives; report B3 as a ceiling
Construct	'Unauthorized action' and 'harm' are simulated by the experimenter; real harm is not measured	State this explicitly; M1–M2 count policy violations, not harm
Construct	Attribution accuracy (M5) is measured against a ground truth that exists only in simulation	Report as an upper bound on real attribution quality
External	Simulated network, simplified adversary, synthetic workload; no real agents or codebases	No generalization claimed; deployed study is future work
External	Results depend on AetherMesh's own unverified simulation assumptions	Report parameters; vary network model
Conclusion	Few scenario classes; thresholds are proposals	Report all outcomes, including negative controls
Literature	Access-control literature was not reviewed; prior art may already cover R1–R6 for centralized settings	Acknowledged; a review is needed before any strong claim (Q2)

7.5 Open Questions (Q1–Q20)

The following questions are open. Appendix C gives, for each, the failure condition it bears on and what would help resolve it.

ID	Question	Bears on
Q1	Is governance a bounded extension of AetherMesh's membership and key management?	F1
Q2	Which requirements beyond R1–R6 are needed, and which prior access-control work already covers them?	F1
Q3	Can accountability records attribute actions while protecting the privacy of agents and their principals?	F5
Q4	Who sets approval thresholds for policy changes, and how?	F4
Q5	How should time-dependent or retroactive policies be handled?	F4
Q6	What precedence rules beyond 'restrictive prevails' are appropriate for conflicting policies?	F4
Q7	Can delegation-chain validity and policy consistency be encoded in a proof language, and what assumptions remain?	F2
Q8	Does F3 hold in centralized or hybrid deployments with an orchestrator?	F3
Q9	What are the semantics of a hybrid between causal and eventual validity with quarantine?	F4
Q10	How are roots of authority established, rotated, and agreed without a central authority?	F4
Q11	What Sybil resistance suits open agent populations?	F4
Q12	How can expiry be enforced when logical time is reported by the actor?	F4
Q13	What is the distribution, especially the tail, of the revocation exposure window?	F5
Q14	How do governance fields map onto AetherGit changeset identifiers?	F1
Q15	What authority should automated remediation agents hold?	F3
Q16	How is circularity avoided between verifying policies and governing verification?	F2
Q17	What are the cost and depth limits of chain verification at scale?	F5
Q18	What is the role of humans in dispute escalation in a decentralized setting?	F4

ID	Question	Bears on
Q19	How are agent identities mapped to responsible organizations or people?	F5
Q20	How should orchestration and runtime assurance be tested against analogous failure conditions?	F3

Limitations of this section. The evaluation is a design, not a result. Thresholds are the author's proposals. The scenario classes are not exhaustive and the simulation platform inherits the limitations of the AetherMesh testbed.

8. Conclusion and Future Work

8.1 Summary of Findings

This paper asked whether authority, policy, accountability, revocation, and dispute resolution constitute a genuine gap in the Aether family. It stated five failure conditions and a decision rule, compared three candidate capabilities, specified a five-layer governance architecture, and stated two propositions with proof sketches. The following are findings in the sense of conclusions reached by argument, not results of experiment.

1. **Fragments of governance exist inside the existing dimensions** (key registry in AetherMesh; review roles in AetherProof), but scoped delegation, revocation with dependents, and dispute handling are not specified in any of them (observation, Section 5.8).
2. **A determinism problem is created by naive authority checks** in a decentralized setting, and can be avoided by having actions cite the revocation frontier and policy version they relied on (Proposition 1, sketch).
3. **Revocation can be recorded without coordination** because validity is antitone in the revocation set (Proposition 2, sketch), **but its timeliness cannot**: an exposure window remains (observation, Section 5.3.2).
4. **F1–F4 were not falsified by argument**, with residual weaknesses: the possibility that governance is a bounded extension of AetherMesh (F1), the reliance on an unformalized decision/anchor distinction (F2), the dependence of F3 on the absence of an orchestrator, and the root-of-authority and exposure-window problems (F4).
5. **F5 is untested**, and the evaluation design of Section 7 specifies how it would be tested and what would count as failure.

8.2 What the Evidence Supports

The evidence, which is argument, supports the modest statement that there is a coherent set of governance requirements not fully specified by the existing records, and that a decentralized design for them can be stated without an evident contradiction with the family's determinism requirement. It also supports the narrower claim that any such design must accept an exposure window and an explicit trust anchor. These are claims about specification, not about performance or deployment.

8.3 What the Evidence Does Not Support

- That governance is necessary for the family, or that a fifth dimension is warranted. F5 is untested and F1 remains open.
- That AetherGovern is the strongest of the three candidates. The other two were not assessed against the failure conditions.

- That any security property holds in practice. Propositions 1 and 2 concern determinism and monotonicity, not resistance to attack.
- That the exposure window is small. H1 is a hypothesis resting on unverified results from another preprint.
- That the design is implementable at acceptable cost. No implementation or measurement exists.
- That the architecture solves accountability in a legal or organizational sense. It provides records, not responsibility.

8.4 Future Work

Direction	Description	Bears on
Independent review of F1–F5	Invite a reviewer unconnected with the Aether preprints to attack the conditions, the requirement list, and the thresholds	All
Prior-art review	Systematic review of access-control, capability, and trust-management literature for R1–R6	F1, Q2
Formal encoding	Encode chain validity and policy consistency in a proof language; record what must be assumed	F2, Q7
Mechanized check of Proposition 1	Model the causal-authority predicate within a formalization of AetherMesh's admission and test determinism exhaustively on small instances	F4
Pre-registered simulation	Run the evaluation of Section 7 on an extended AetherMesh testbed	F5
Assess the other candidates	Apply analogous failure conditions to orchestration and runtime assurance, then compare	Q20
Decide project structure	After the above, choose single, dual, or triple structure, or decline all three	—

8.5 A Modest Conclusion

AetherGovern is a candidate, not a result. The investigation found that the question 'who is authorized to do what, under which policy?' is not answered by the four existing Aether preprints, and that a decentralized answer is not ruled out by the determinism requirement that governs the family. It also found that the honest answer carries two costs that cannot be designed away: an exposure window after revocation and a dependence on declared trust anchors. Whether these costs are worth bearing for a separate dimension, as opposed to an extension of an existing one, is exactly what remains to be tested. If the evaluation of Section 7 shows no scenario in which the family with governance outperforms the best simple baseline, the appropriate conclusion will be that governance is a topic within AetherMesh or AetherProof, not a dimension of its own, and that conclusion would be a useful result.

Limitations of this section. The conclusions reached are provisional and rest on argument by a single author. The final project structure is deliberately not decided.

Appendix A — Notation

Symbol	Meaning (definition)
F1–F5	Failure conditions of Section 3.2
R1–R6	Candidate governance requirements of Section 3.2
M1–M9	Evaluation metrics of Section 7.1
B1–B5	Baselines of Section 7.2
S1–S6	Scenario classes of Section 7.3
P1–P8	Design principles of Section 5.6
Q1–Q20	Open questions of Section 7.5 and Appendix C
H1	Hypothesis on the revocation exposure window (Section 5.3.2)
c = (issuer, subject, scope, constraints, expiry, parent, signature)	Delegation certificate
scope(c)	The set of (action, resource) pairs authorized by certificate c
$S' \subseteq S$	Scope inclusion: S′ confers no authority beyond S
F, F′	Revocation frontiers (sets of revocation records, identified by hash)
valid(c, F)	Chain c is valid at frontier F (Section 5.3)
⊔	Join of a join-semilattice (as in AetherMesh); for revocation sets, set union
β, t	Gossip transmission constant and step count, as in AetherMesh's Definition 3
A1–A6 (AetherMesh)	Assumptions of the AetherMesh preprint; A6 is admission determinism

Appendix B — The Five Failure Conditions (F1–F5) in Detail

Each condition is given in a fixed format: the claim under test; why it is a reasonable test; the procedure; what would falsify it; what would count only as weak support; the current provisional status; and the residual weakness. The conditions were proposed by the author and were chosen before the assessment of Section 5.8.

F1 — Non-redundancy

Item	Content
Claim under test	Some authorization requirement among R1–R6 is not met by the specified mechanisms of the four existing dimensions, even with one bounded, localized extension to a single dimension.
Why this test	If a requirement is already met, or would be met by a small extension of one dimension, a new dimension is not warranted: its content would be a feature.
Procedure	(1) For each requirement, locate the specification in each existing preprint that addresses it. (2) Where none is found, estimate the extension needed to the closest dimension and judge whether it is bounded and localized. (3) Record the judgement and the reasoning.
Falsified if	Every requirement is covered, or each uncovered requirement is covered by a bounded extension of one dimension.
Weak support only	A mapping showing that requirements are uncovered does not itself show that the extension is unbounded.
Provisional status	Not falsified for R1, R3, R5; partial coverage of R2, R4, R6.
Residual weakness	The boundedness judgement is subjective. The minimal interface (Section 6.6) may be exactly such a bounded extension of AetherMesh.

F2 — Orthogonality to verification

Item	Content
Claim under test	Authorization decisions, and the trust anchors from which they derive, cannot be reduced to proof obligations in AetherProof without loss.
Why this test	If governance is a special case of verification, it should be folded into AetherProof.
Procedure	(1) Attempt to encode chain validity, policy consistency, and revocation as propositions and obligations. (2) List every assumption that remains unverified after encoding. (3) Ask whether any remaining assumption is a governance fact (who may declare a root, policy, or specification).
Falsified if	The encoding is complete, with no remaining unverified assumption that concerns authority.
Weak support only	That verification tools can check chain validity given anchors is expected and does not falsify F2.
Provisional status	Not falsified, narrowly: chain validity is encodable; anchors are not.
Residual weakness	The decision/anchor distinction may be rejected by a critic. No formal encoding was attempted.

F3 — Boundary with orchestration

Item	Content
Claim under test	In a decentralized setting, governance is distinct from orchestration.
Why this test	If governance reduces to task assignment plus access control located in an orchestrator, it belongs with orchestration.
Procedure	(1) Remove the orchestrator from a scenario. (2) Check whether authority questions remain. (3) Check whether any authority question is answerable only by assignment.
Falsified if	Every authority question is answered by assignment, or by an access-control feature of a central orchestrator.
Weak support only	Showing that an orchestrator can enforce policy shows only that it can, not that it must.
Provisional status	Not falsified in the decentralized setting; likely to fail in centralized deployments.
Residual weakness	Dependence on the premise that no orchestrator is present. Hybrid deployments are unassessed.

F4 — Decentralizability

Item	Content
Claim under test	Governance can be specified without a trusted central authority and without violating the determinism required by AetherMesh.
Why this test	A governance design that requires a central authority or strong consistency contradicts the premise of the family.
Procedure	(1) Specify each layer's core operation. (2) Check each against the determinism requirement. (3) Search for any operation whose correctness needs agreement among all nodes.
Falsified if	An operation needed for the layer's purpose that cannot be done without a central authority or strong consistency.
Weak support only	A decentralized design that is merely plausible is weak support, because plausibility is cheap.
Provisional status	Not falsified, with explicit residuals (Propositions 1 and 2; exposure window; roots).
Residual weakness	Timeliness of revocation depends on coordination that the design lacks; roots are a central dependency in trust terms.

F5 — Observable benefit

Item	Content
Claim under test	There exist scenario classes in which the family with governance outperforms the family without it, on a pre-specified metric.
Why this test	A dimension that provides no observable benefit is not a dimension, however coherent its specification.
Procedure	The pre-registered simulation of Section 7.3, with the thresholds there.
Falsified if	No scenario class meets the F5 criterion of Section 7.3.
Weak support only	Argument and illustration are not evidence for F5.
Provisional status	Untested.
Residual weakness	Every conclusion about whether to proceed depends on it.

Appendix C — Open Questions Q1–Q20

For each question: the failure condition it bears on and what would help resolve it.

ID	Question	Bears on	What would help resolve it
Q1	Is governance a bounded extension of AetherMesh's membership and key management?	F1	Independent judgement of the minimal interface by a reviewer; an implementation sketch within AetherMesh's registry
Q2	Which requirements beyond R1–R6 are needed, and which prior access-control work already covers them?	F1	Systematic literature review of access-control, capability, and trust-management work; requirement elicitation from practitioners
Q3	Can accountability records attribute actions while protecting the privacy of agents and their principals?	F5	Study of attribution schemes that separate agent identity from principal identity; privacy analysis
Q4	Who sets approval thresholds for policy changes, and how?	F4	Comparison of threshold schemes for collective approval; analysis of capture risks
Q5	How should time-dependent or retroactive policies be handled?	F4	Specification of time semantics for policies; study of retroactive-effect cases
Q6	What precedence rules beyond 'restrictive prevails' are appropriate for conflicting policies?	F4	Catalogue of conflict types and rules; simulation of rule outcomes
Q7	Can delegation-chain validity and policy consistency be encoded in a proof language, and what assumptions remain?	F2	Encoding attempt in a proof assistant or model checker; list of residual assumptions
Q8	Does F3 hold in centralized or hybrid deployments with an orchestrator?	F3	Scenario analysis with a central orchestrator; hybrid deployment study
Q9	What are the semantics of a hybrid between causal and eventual validity with quarantine?	F4	Formal semantics of quarantine and rollback within an inflationary state model
Q10	How are roots of authority established, rotated, and agreed without a central authority?	F4	Survey of root-of-trust and threshold-key designs; analysis under Byzantine bounds
Q11	What Sybil resistance suits open agent populations?	F4	Evaluation of identity-cost mechanisms; adversary model with Sybil identities
Q12	How can expiry be enforced when logical time is reported by the actor?	F4	Design of verifiable time sources or logical-time commitments; analysis of actor-reported time
Q13	What is the distribution, especially the tail, of the revocation exposure window?	F5	Pre-registered simulation (M3); sensitivity to network model
Q14	How do governance fields map onto AetherGit changeset identifiers?	F1	Schema comparison with AetherGit's record identifiers; compatibility analysis
Q15	What authority should automated remediation agents hold?	F3	Authority model for automated agents; separation-of-duty analysis

DOI: https://doi.org/10.5281/zenodo.24054524

ID	Question	Bears on	What would help resolve it
Q16	How is circularity avoided between verifying policies and governing verification?	F2	Dependency analysis between policy verification and governed verification; grounding in a root
Q17	What are the cost and depth limits of chain verification at scale?	F5	Complexity analysis and measurement of chain verification at scale
Q18	What is the role of humans in dispute escalation in a decentralized setting?	F4	Case studies of escalation in decentralized communities; design of human roles
Q19	How are agent identities mapped to responsible organizations or people?	F5	Legal and organizational analysis of agent-to-principal mappings
Q20	How should orchestration and runtime assurance be tested against analogous failure conditions?	F3	Apply F1–F5 analogues to the other two candidates; comparison under one method

DOI: https://doi.org/10.5281/zenodo.24054524

Appendix D — Glossary

Term	Meaning
Accountability	After-the-fact ability to determine which agents were responsible for an admitted action, from recorded evidence.
Action	An operation by an agent on a resource, such as commit, merge, delete, deploy, or key rotation.
Agent	An autonomous software actor, identified by a key, that proposes changes.
Attenuation	The property that a delegated scope is a subset of the delegator's scope.
Authority	The right of an agent to perform actions within a scope, conferred by a chain of certificates from a root.
Authorization	Determining whether an agent may perform a given action.
Causal closure	An item together with all items it depends on causally (its ancestors).
Causal-authority admission	Evaluating authority at the revocation frontier cited by the action (Section 5.3.1).
Chain	A sequence of delegation certificates from a root to an agent.
Delegation certificate	A signed record conferring a scope on a subject.
Dimension	A functional concern addressed by one Aether preprint; not a hierarchical layer.
Dispute resolution	Resolving conflicting authority claims by rule or escalation.
Eventual consistency	Replicas that receive the same updates converge to the same state.
Exposure window	Interval in which a node may admit actions citing authority that has been revoked elsewhere.
Failure condition	A condition whose satisfaction shows that the candidate does not warrant a dimension of its own.
Falsification-first	Stating what would show a hypothesis false before seeking supporting evidence.
Governance	The rules and mechanisms deciding who may act, under which policy, with what accountability.
Hypothesis	A claim thought plausible and not established.
Join-semilattice	A set with a commutative, associative, idempotent join; the state space of a CRDT.
Minimal interface	The smallest interface giving scoped delegation and revocation (Section 6.6).
Observation	A claim about the literature or records that a reader can check against the cited source.
Orchestration	Deciding what agents do next: task decomposition, assignment, scheduling.
Orthogonality	Independence of two concerns at the level of the question asked.
Policy	A rule governing authorization, with a version and approval record.
Revocation	Withdrawal of authority previously conferred.
Revocation frontier	The set of revocation records an actor had seen when acting.
Root of authority	A declared trust anchor from which chains start.
Runtime assurance	Monitoring and constraining agents while they execute.
Scope	The set of actions and resources an authority covers.
Trust anchor	An assumption about identity or authority held without derivation from other records.

References

Almeida, P. S., Shoker, A., & Baquero, C. (2018). Delta state replicated data types. *Journal of Parallel and Distributed Computing, 111*, 162–173.

Bailis, P., & Ghodsi, A. (2013). Eventual consistency today: Limitations, extensions, and beyond. *Communications of the ACM, 56*(5), 55–63.

Baladi, S. (2026a). *AetherGit: Scalable Source Control for Autonomous AI Agents*. Zenodo. https://doi.org/10.5281/zenodo.23966896

Baladi, S. (2026b). *AetherFlow: Automated Post-Merge Verification and Dynamic Conflict Resolution in Multi-Agent Programming Environments*. Zenodo. https://doi.org/10.5281/zenodo.23979129

Baladi, S. (2026c). *AetherProof: Autonomous Formal Verification and Invariant Synthesis for Multi-Agent Software Ecosystems*. Zenodo. https://doi.org/10.5281/zenodo.24013124

Baladi, S. (2026d). *AetherMesh: Decentralized Peer-to-Peer Agent Synchronization and State CRDTs for Autonomous Multi-Agent Systems*. Zenodo. https://doi.org/10.5281/zenodo.24038038

Bernstein, D. J., Duif, N., Lange, T., Schwabe, P., & Yang, B.-Y. (2012). High-speed high-security signatures. *Journal of Cryptographic Engineering, 2*(2), 77–89.

Brousseau, E., & Glachant, J.-M. (Eds.). (2008). *New Institutional Economics: A Guidebook*. Cambridge University Press.

Castro, M., & Liskov, B. (1999). Practical Byzantine fault tolerance. In *Proc. 3rd USENIX Symposium on Operating Systems Design and Implementation (OSDI)* (pp. 173–186).

DeNardis, L. (2014). *The Global War for Internet Governance*. Yale University Press.

Demers, A., Greene, D., Hauser, C., et al. (1987). Epidemic algorithms for replicated database maintenance. In *Proc. 6th ACM Symposium on Principles of Distributed Computing (PODC)* (pp. 1–12).

Douceur, J. R. (2002). The Sybil attack. In *Peer-to-Peer Systems (IPTPS 2002)*, LNCS 2429 (pp. 251–260). Springer.

Hardin, G. (1968). The tragedy of the commons. *Science, 162*(3859), 1243–1248.

Jelasity, M., Montresor, A., & Babaoglu, O. (2005). Gossip-based aggregation in large dynamic networks. *ACM Transactions on Computer Systems, 23*(3), 219–252.

Karp, R., Schindelhauer, C., Shenker, S., & Vöcking, B. (2000). Randomized rumor spreading. In *Proc. 41st IEEE Symposium on Foundations of Computer Science (FOCS)* (pp. 565–574).

Kleppmann, M., & Beresford, A. R. (2017). A conflict-free replicated JSON datatype. *IEEE Transactions on Parallel and Distributed Systems, 28*(10), 2733–2746.

Lamport, L., Shostak, R., & Pease, M. (1982). The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems, 4*(3), 382–401.

Lessig, L. (1999). *Code and Other Laws of Cyberspace*. Basic Books.

Merkle, R. C. (1988). A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO '87*, LNCS 293 (pp. 369–378). Springer.

North, D. C. (1990). *Institutions, Institutional Change and Economic Performance*. Cambridge University Press.

Ostrom, E. (1990). *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press.

Schneier, B. (2012). *Liars and Outliers: Enabling the Trust that Society Needs to Thrive*. Wiley.

Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, LNCS 6976 (pp. 386–400). Springer.

Weitzner, D. J., Abelson, H., Berners-Lee, T., Feigenbaum, J., Hendler, J., & Sussman, G. J. (2008). Information accountability. *Communications of the ACM, 51*(6), 82–87.

Williamson, O. E. (1985). *The Economic Institutions of Capitalism*. Free Press.

Note: bibliographic details were checked against the author's supplied list and the author's knowledge of the literature; three entries (Weitzner et al., Brousseau and Glachant, Schneier) were additionally confirmed to exist by web search. Volumes, pages, and venues should be confirmed against the publishers' records before submission. The four companion papers are cited from their Zenodo records and have not been peer reviewed.

Research Status Statement

Item	Status
Document type	Research article: structured investigation of an architectural hypothesis
Version / record	0.1.0, November 2026 · DOI 10.5281/zenodo.24054524
Project status	Not a committed project; final structure (single, dual, or triple) undetermined
Implementation	None claimed
Real-world deployment	None; none evaluated
Empirical results	None reported
Propositions 1–2	Stated with proof sketches; not machine-checked
F1–F4	Not falsified by argument alone, with residual weaknesses; provisional
F5	Untested
AetherGovern	A candidate for addressing the authorization gap, provided it passes F1–F5; not a confirmed dimension
Review status	Not peer reviewed; the four companion papers are likewise unreviewed preprints

Reconciliation notes

1 **Failure conditions.** The brief named F1–F5 but did not define them. The definitions, the requirement list R1–R6, and the decision rule are the writer's proposals and should be confirmed or replaced by the author.

2 **AetherMesh DOI.** The supplied reference list gave 10.5281/zenodo.24053492 for AetherMesh, whereas the DOI supplied for the AetherMesh paper was 10.5281/zenodo.24038038. The latter is used throughout; the author should confirm which is correct.

3 **Reference check.** All 25 entries correspond to the intended publications. Corrections made: the AetherMesh title was expanded to the full title; diacritics restored (Vöcking, Preguiça); the Weitzner et al. author list completed; entries alphabetized and Baladi entries lettered 2026a–d.

4 **Figures.** The five supplied figures were redrawn under their supplied filenames. Changes: Figure 2 now states that its ordering is conceptual dependency (the original arrow read 'bottom to top' while Figure 1 says the items are not hierarchical); the revocation claim in Figure 4 is qualified by 'once the revocation has reached the verifying node'; the verification expression in Figure 4 now includes attenuation, expiry, and a revocation check; decorative icons with rendering artefacts were removed. Figures 6 and 7 are supplementary.

5 **Layers versus dimensions.** The supplied text calls the five items 'dimensions' and the governance sub-components 'layers'; this usage is kept.

6 **Format.** The brief requested Markdown; this record is a PDF, as requested separately.

Author information. Samir Baladi, Interdisciplinary AI Research, Ronin Institute / Rite of Renaissance. Email: gitdeeper@gmail.com. ORCID: 0009-0003-8903-0029. **Citation:** Baladi, S. (2026). AetherGovern: A Falsification-First Investigation of Governance in the Aether Family. Zenodo. https://doi.org/10.5281/zenodo.24054524